



UNIVERSIDAD AUTONOMA GABRIEL RENE MORENO

FACULTAD DE INTEGRAL DEL CHACO

INGENIERIA INFORMATICA - INGENIERIA EN SISTEMAS

Implementación una plataforma para la automatización de la mensajería institucional y comercial mediante la integración de agentes de autorespuesta en canales de mensajería

Materia: *Taller De Grado 1 - INF511*

Docente: Msc. Ing. Jose Manuel Colque Paulasanz

Estudiantes:

Núñez Ardaya Melissa Shanner - 222106301

Pozo Solíz Marthel Pedro - 222106867

Gestión 1-2026

Camiri, 26 de abril de 2026

Índice

Perfil	1
Introducción	1
Descripción del dominio del problema	1
Antecedentes	2
Justificación	4
Objetivo general	5
Objetivos específicos	5
Alcance	5
Metodología de desarrollo	6
Cronograma y gestión del flujo de trabajo	7
Marco teórico	8
Conceptos clave del proyecto	8
Tecnologías empleadas	9
Análisis	10
Requerimientos funcionales	10
Requerimientos no funcionales	10
Reglas de negocio	10
Actores del sistema	11
Casos de uso principales	11
Alcance del MVP	11
Restricciones y supuestos	11
Criterios de aceptación	12
Riesgos técnicos y operativos	12
Metodología de desarrollo y validación	12
Enfoque metodológico general	12
Kanban como metodología principal	12
Estructura del tablero Kanban en Taiga	13
Integración de Lean Startup	13
Ciclo Construir-Medir-Aprender	13
Hipótesis de valor y crecimiento	13

Métricas Kanban-Lean	14
Registro de experimentos e incidencias críticas	14
Criterios para pivotar o perseverar	14
Diseño	14
Arquitectura del sistema	14
Autenticación y seguridad	15
Principios de diseño	15
Convenciones de código	15
Configuración	15
Control de versiones	15
Arquitectura backend	15
Topología de despliegue	17
Entornos de ejecución	17
Escalabilidad	17
Seguridad	17
Monitoreo y recuperación	17
Diseño de base de datos	17
Modelo de datos	17
Convenciones de datos	17
Diseño de API e integraciones	18
Diseño de experiencia de usuario	18
Desarrollo	18
Estrategia de desarrollo incremental	18
Funcionalidades del MVP	18
Validación, métricas y aprendizaje	18
Estrategia de validación técnica y funcional	18
Registro de experimentos y aprendizajes	19
Discusión de resultados	19
Conclusiones y recomendaciones	19
Conclusiones generales	19
Aportes metodológicos del proyecto	19

Limitaciones del trabajo 19

Recomendaciones 19

Líneas de trabajo futuro 19

Índice de figuras

1. Cronograma general de actividades del proyecto 7

Índice de cuadros

1. Endpoints principales de la API 18

Perfil

Introducción

En la actualidad, la comunicación digital mediante aplicaciones de mensajería como WhatsApp y Telegram se ha consolidado como un canal principal de interacción entre instituciones, negocios y usuarios. No obstante, gran parte de estas interacciones aún se gestionan de forma manual, lo que genera tiempos de respuesta elevados, sobrecarga operativa y dificultades para mantener la consistencia de la información compartida.

En este contexto, diversos trabajos recientes han demostrado el potencial de los chatbots para mejorar la atención y automatizar consultas frecuentes en entornos educativos y comerciales. Los antecedentes revisados evidencian resultados favorables en términos de eficiencia, trazabilidad y experiencia del usuario cuando se integran agentes conversacionales con plataformas de mensajería y arquitecturas web escalables. A partir de esa base, el presente proyecto propone una solución tipo *Software como Servicio (SaaS)* orientada a centralizar la comunicación institucional y comercial, automatizar respuestas recurrentes y facilitar la administración de interacciones desde una interfaz web. Su importancia radica en aportar una alternativa tecnológica práctica para optimizar procesos de atención, reducir carga del personal y mejorar la calidad del servicio ofrecido a los usuarios.

De manera general, el trabajo se sustenta en antecedentes de implementación de asistentes virtuales, se justifica por la necesidad de modernizar la gestión de consultas digitales y delimita su alcance al desarrollo de una plataforma con integración multicanal, configuración de respuestas automáticas, registro de conversaciones y monitoreo básico. En conjunto, la propuesta busca contribuir a una comunicación más eficiente, oportuna y sostenible en organizaciones que dependen de la atención digital.

Descripción del dominio del problema

El dominio del problema del proyecto se ubica en la **comunicación institucional y comercial digital**, específicamente en la atención de consultas de usuarios a través de canales de mensajería instantánea. De forma general, este dominio surge de la transformación de los procesos de atención tradicionales (presenciales, telefónicos o por correo) hacia interacciones digitales de alta inmediatez, donde las personas esperan respuestas rápidas, claras y disponibles en todo momento.

En una perspectiva cronológica, durante los primeros años de digitalización de servicios predominó el uso de páginas web, correos electrónicos y formularios en línea como mecanismos principales de contacto. Aunque estos medios permitieron ampliar cobertura, no resolvieron completamente los problemas de tiempos de espera, seguimiento de casos y estandarización de respuestas. Con la masificación de smartphones y aplicaciones de chat, la mensajería instantánea pasó a ocupar un rol central en la relación

entre instituciones y usuarios, desplazando progresivamente otros canales por su accesibilidad y menor fricción en el uso.

En esta etapa reciente, plataformas como WhatsApp y Telegram se consolidaron como espacios de atención, soporte y difusión de información. Sin embargo, su adopción acelerada también evidenció limitaciones operativas: incremento de consultas repetitivas, dependencia de atención manual, saturación de personal y dificultades para sostener calidad homogénea en las respuestas. Esta realidad se observa tanto en entornos comerciales como en espacios académicos e institucionales, donde la demanda informativa incluye horarios, requisitos, estados de trámite, fechas clave, costos, disponibilidad de servicios y orientación general.

Desde lo particular, el objeto de estudio de este proyecto se centra en el **diseño e implementación de una plataforma SaaS de autorespuesta** para organizaciones, negocios e instituciones que requieren centralizar conversaciones de WhatsApp y Telegram, automatizar consultas frecuentes y mantener trazabilidad de interacción. Por tanto, el problema específico no es solo responder mensajes", sino **optimizar integralmente el proceso de comunicación digital** mediante una solución que combine inmediatez, consistencia informativa, escalabilidad y capacidad de gestión.

En síntesis, la realidad actual del dominio evidencia una transición: de modelos manuales y fragmentados hacia esquemas automatizados e integrados. La brecha identificada se encuentra en la necesidad de soluciones accesibles que, además de automatizar, permitan administrar de forma unificada la comunicación y sostener calidad de servicio en contextos de alta demanda. Esa brecha define y justifica el alcance del presente trabajo.

Antecedentes

(Lino Cruz & Paladines Morán, 2023), *Implementación de una plataforma de chatbot gestionado por WhatsApp y Telegram para su comercialización en la empresa Innovus Software*, desarrollada en la Universidad Estatal del Sur de Manabí (UNESUM), en Jipijapa.

Este trabajo se centra en la mejora de la atención al cliente mediante un servicio de mensajería automática, orientado a responder consultas frecuentes de forma inmediata y precisa. A nivel metodológico, el estudio emplea revisión bibliográfica, análisis documental, enfoque histórico-lógico y observación del comportamiento de las consultas realizadas por los clientes.

Entre sus principales hallazgos, se reporta que una parte importante de las preguntas recibidas por la empresa son repetitivas y objetivas, lo que justifica la automatización del proceso de respuesta. La propuesta técnica se apoya en librerías (WhatsApp Cloud API) para la escucha de eventos de mensajería y en un algoritmo de secuencias programadas para identificar mensajes y generar respuestas automáticas.

Como resultado, la investigación concluye que la implementación de una plataforma de chatbot en canales como WhatsApp y Telegram contribuye a reducir costos operativos, incrementar la productividad y

habilitar un servicio tecnológicamente viable para su comercialización a terceros. Este antecedente respalda la pertinencia del presente proyecto, especialmente en su enfoque de automatización de la comunicación para organizaciones, tiendas y negocios con alta demanda de atención digital.

(Vargas & Braga, s.f.), en su trabajo *Implementação de assistente virtual via WhatsApp para aprimorar a comunicação no IFC – Campus Sombrio*, presentan el desarrollo de un asistente virtual orientado a fortalecer la comunicación institucional y la difusión de información oficial del campus. La propuesta se enfoca en responder dudas generales de los usuarios (por ejemplo, cursos, docentes y ubicación), y se enmarca como una investigación tecnológica basada en la construcción de un artefacto funcional.

Técnicamente, el sistema fue implementado con JavaScript/TypeScript e integrado con la API Baileys para la conexión con WhatsApp. Como resultado, reportan un asistente capaz de entregar respuestas simples de forma más rápida que la búsqueda tradicional en redes sociales o en la web, destacando además su potencial de ampliación con nuevas funcionalidades.

(Zarabanda Gelves, 2024) plantea el desarrollo de un chatbot interactivo para el proyecto curricular de Ingeniería Telemática y Sistematización de Datos de la Universidad Distrital Francisco José de Caldas, con el propósito de mejorar la comunicación entre estudiantes y personal administrativo por medio de WhatsApp. La propuesta contempla funcionalidades orientadas a la gestión de trámites de radicación de anteproyectos, notificación de fechas clave, actualización de enlaces para documentación requerida y atención de preguntas frecuentes. A nivel arquitectónico, el diseño prioriza modularidad y escalabilidad: los mensajes son gestionados con Baileys y procesados en un servidor Node.js, que a su vez puede consultar MariaDB y APIs externas para construir respuestas adecuadas. Además, el proyecto adopta metodología SCRUM, con roles definidos (Product Owner, Scrum Máster y equipo de desarrollo) y trabajo organizado en sprints con entregables progresivos desde la configuración inicial hasta el despliegue y mantenimiento.

(Elsen & Muhamad, 2025) desarrollan una aplicación web de catálogo electrónico de productos hiyab integrada con un chatbot de WhatsApp para fortalecer la atención al cliente en una MIPYME del sector moda musulmana en Indonesia. El estudio atiende problemas de gestión manual de inventario y baja capacidad de respuesta administrativa, proponiendo un sistema que permite consultar detalles de productos, precios, stock en tiempo real y promociones, además de gestionar pedidos. El chatbot, implementado con la API Baileys, responde consultas frecuentes como ubicación de tienda y valoraciones de productos, reduciendo carga operativa. La validación reporta resultados satisfactorios de usabilidad (SUS 71.3 para no clientes y 73.5 para clientes), lo que respalda la pertinencia de integrar catálogos web y agentes conversacionales para mejorar eficiencia operativa y experiencia de usuario.

(Vega Pacheco, 2023), en su tesis de grado de la Universidad Mayor de San Andrés, analiza el uso de procesamiento de lenguaje natural con modelos inteligentes para la atención de consultas mediante redes

sociales. El estudio describe la aplicación de redes neuronales artificiales y arquitecturas Transformer para mejorar la comprensión del lenguaje en chatbots, con el objetivo de ofrecer interacciones más naturales y efectivas con los usuarios. Como caso aplicado, propone un chatbot para el gimnasio CrossFit capaz de informar sobre horarios, precios, actividades y recomendaciones de entrenamiento y nutrición. Entre los resultados comparativos, reporta mejor desempeño del modelo de red neuronal artificial frente a GPT-2 y GPT-3 dentro de la metodología empleada. Este antecedente aporta evidencia sobre la pertinencia de combinar técnicas de IA y PLN para fortalecer canales de atención digital.

Justificación

El crecimiento del uso de plataformas de mensajería instantánea ha transformado la forma en que las organizaciones interactúan con sus usuarios. Actualmente, canales como WhatsApp y Telegram se han convertido en medios fundamentales para la atención al cliente, el soporte y la gestión de consultas informativas. Sin embargo, muchas instituciones y negocios continúan gestionando estas interacciones de forma manual, lo que provoca retrasos en la respuesta, sobrecarga del personal y dificultades para mantener la consistencia de la información.

Desde la realidad observada en contextos institucionales y comerciales, se identifica una demanda recurrente de consultas sobre horarios, requisitos, estados de trámite, precios, ubicación y orientación general. Estas consultas, al repetirse con frecuencia, consumen tiempo operativo y limitan la capacidad del personal para atender casos de mayor complejidad. Además, cuando la atención se realiza en canales no integrados, se incrementa la duplicación de tareas y se dificulta el seguimiento ordenado de conversaciones. Otro aspecto crítico es la variabilidad en los tiempos de respuesta, especialmente fuera del horario laboral, lo que impacta en la experiencia del usuario y en la percepción de calidad del servicio. En este escenario, la falta de automatización y de trazabilidad afecta tanto la eficiencia operativa como la continuidad de la comunicación.

En este contexto, el desarrollo de una plataforma basada en *Software como Servicio (SaaS)* para automatizar la atención mediante bots conversacionales representa una solución tecnológica viable y necesaria. La automatización de consultas frecuentes permite reducir carga operativa, mejorar la oportunidad de respuesta y liberar recursos humanos para interacciones que requieren criterio especializado. Asimismo, la integración de múltiples canales en una única plataforma favorece la centralización de conversaciones, el registro del historial, el monitoreo del servicio y una gestión más ordenada de la información. Por tanto, la justificación del proyecto se sustenta en una brecha real entre la demanda comunicacional y la capacidad de respuesta manual, y en la necesidad de implementar una solución que optimice los procesos de atención digital con mayor consistencia, continuidad y calidad.

Objetivo general

Implementar una plataforma bajo el modelo *Software como Servicio (SaaS)* para la automatización de la mensajería institucional y comercial, con el fin de optimizar los tiempos de respuesta, mejorar la atención a usuarios mediante la integración agentes de autorespuestas en canales como WhatsApp y Telegram.

Objetivos específicos

- Analizar las necesidades de comunicación digital de organizaciones y negocios que utilizan plataformas de mensajería instantánea para la atención a usuarios.
- Diseñar la arquitectura de una plataforma que permita integrar y gestionar conversaciones provenientes de WhatsApp y Telegram en un entorno centralizado.
- Desarrollar un sistema de agentes de autorespuesta capaz de identificar consultas frecuentes y generar respuestas automáticas de forma eficiente.
- Implementar un panel de administración que permita configurar respuestas, monitorear conversaciones y gestionar la interacción con los usuarios.
- Validar el funcionamiento de la plataforma mediante pruebas de integración y verificación del correcto envío y recepción de mensajes automatizados en entornos controlados

Alcance

El presente proyecto se desarrolla en el contexto de organizaciones de la región hispanohablante que gestionan atención digital mediante mensajería instantánea, con énfasis en instituciones educativas, unidades administrativas y negocios que requieren comunicación frecuente con usuarios finales. Dentro del dominio general de comunicación institucional digital, la región de trabajo del proyecto se delimita a entornos donde WhatsApp y Telegram constituyen canales principales de contacto operativo.

En términos organizacionales, el alcance involucra principalmente áreas de **atención al usuario/cliente, secretaría o administración, coordinación académica o comercial** (según el tipo de organización) y **soporte técnico o sistemas**. Estas áreas participan en la configuración de respuestas, validación de información oficial, seguimiento de conversaciones y supervisión del servicio automatizado.

Desde la perspectiva funcional, el proyecto contempla el diseño e implementación de los siguientes subsistemas.

- **Subsistema de integración de canales:** conexión y recepción/envío de mensajes en WhatsApp y Telegram.
- **Subsistema de gestión conversacional:** reglas de autorespuesta, detección de consultas frecuentes y derivación de casos no resueltos.

- **Subsistema de administración:** panel web para configuración de respuestas, gestión de usuarios y control de parámetros operativos.
- **Subsistema de trazabilidad:** registro de historial de conversaciones y monitoreo básico de interacciones.
- **Subsistema de datos y seguridad:** persistencia de información y controles básicos de acceso para resguardar la operación.

Además el proyecto contempla, como parte de su validación, la realización de pruebas funcionales de los subsistemas desarrollados.

Queda fuera del alcance, en esta etapa, la incorporación de analítica avanzada con modelos predictivos, integración con sistemas empresariales complejos (ERP/CRM a medida), atención por voz y despliegues multinodo de alta disponibilidad. En consecuencia, el proyecto se concentra en una solución SaaS funcional, escalable a nivel inicial y orientada a optimizar la comunicación digital en procesos de atención recurrente.

Metodología de desarrollo

Para la planificación y ejecución del proyecto se adopta Kanban como metodología ágil de trabajo. Esta elección responde a la necesidad de gestionar el desarrollo de forma flexible, visual e incremental, considerando que la solución propuesta integra servicios externos de mensajería y puede requerir ajustes continuos durante su implementación. Kanban permite organizar el flujo de actividades, limitar el trabajo en curso y promover la mejora continua del proceso (Anderson, 2010).

A diferencia de marcos ágiles basados en roles formales y ciclos rígidos, en este proyecto Kanban resulta más adecuado porque facilita la priorización continua de tareas sin imponer ceremonias fijas. Esto es especialmente útil en un equipo pequeño de desarrollo, donde se requiere atender cambios funcionales, bloqueos técnicos y validaciones parciales conforme avanza la construcción del producto mínimo viable. La gestión operativa se realiza mediante un tablero Kanban en Taiga, herramienta que permite visualizar el estado de cada actividad y mantener trazabilidad sobre el avance del proyecto (Taiga Agile, 2026). En dicho tablero se definieron seis columnas de trabajo: **New**, **Ready**, **In Progress**, **Ready for Test**, **Done** y **Archived**. Este flujo permite distinguir con claridad las tareas recién registradas, aquellas listas para ser atendidas, las actividades en ejecución, los elementos preparados para validación funcional, los trabajos concluidos y los ítems cerrados para su conservación histórica.

Adicionalmente, el tablero se estructura en tres *swimlanes* principales: **Panel Web**, **Motor** y **API**, que corresponden a los componentes centrales del sistema. De forma complementaria, se emplean dos *swimlanes* de apoyo: **Pruebas**, destinada al seguimiento de pruebas de funcionamiento, y **Crítico**, utilizada para registrar incidencias, fallos detectados y observaciones prioritarias derivadas del proceso de validación.

Bajo este enfoque, el desarrollo del sistema se estructura en entregas pequeñas y funcionales alineadas con el alcance del MVP. La combinación entre columnas de flujo y *swimlanes* permite organizar el trabajo por componente técnico, controlar el avance de cada tarea y dar seguimiento oportuno a pruebas e incidencias. De esta manera, módulos como la integración con WhatsApp y Telegram, el envío de mensajes, las autorespuestas, el panel de administración y las herramientas de soporte pueden implementarse, validarse y ajustarse progresivamente, asegurando que cada incremento aporte valor verificable al proyecto.

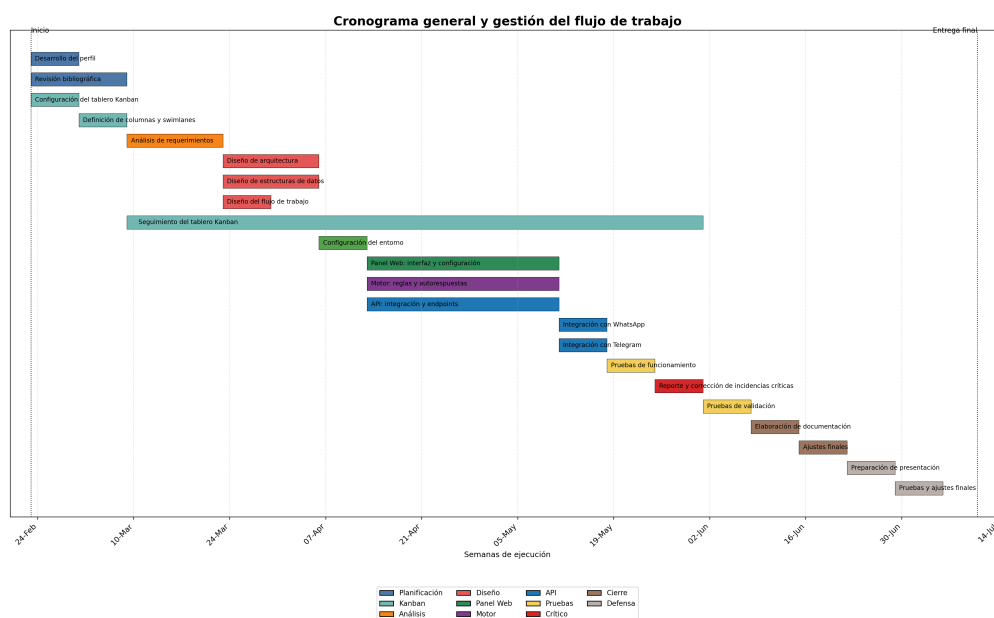
Cronograma y gestión del flujo de trabajo

El cronograma establece la secuencia general de actividades del proyecto desde la etapa de perfil hasta el cierre y preparación de la defensa. De manera complementaria, el seguimiento operativo del trabajo se realiza mediante Kanban en Taiga, lo que permite controlar el avance diario de las tareas sin perder la visión temporal del proyecto.

En este esquema, el cronograma define los periodos de ejecución, mientras que la gestión del flujo de trabajo organiza cada actividad en las columnas New, Ready, In Progress, Ready for Test, Done y Archived. Asimismo, el uso de *swimlanes* para Panel Web, Motor, API, Pruebas y Crítico facilita la identificación de responsabilidades, el seguimiento de validaciones y el tratamiento oportuno de incidencias.

Figura 1

Cronograma general de actividades del proyecto



De este modo, ambos instrumentos se complementan: el cronograma permite planificar hitos y entregas, mientras que Kanban favorece la priorización continua, la detección de bloqueos y la mejora del flujo de trabajo durante la construcción incremental del sistema.

Marco teórico

Conceptos clave del proyecto

Chatbots. Un chatbot puede entenderse como un sistema de interacción que procesa entradas de usuario y genera respuestas según reglas, contexto o modelos de lenguaje. En el ámbito de atención digital, los agentes de autorespuesta se orientan a resolver consultas de baja y media complejidad, dejando los casos especiales a personal humano.

De acuerdo con IBM, un chatbot es un programa informático que simula una conversación humana con un usuario final; no todos los chatbots incorporan inteligencia artificial, pero los chatbots modernos utilizan cada vez más técnicas de IA conversacional, como el procesamiento del lenguaje natural (PLN), para comprender preguntas de los usuarios y automatizar sus respuestas (IBM, 2026).

Software como Servicio (SaaS). Modelo de provisión de software en la nube donde la aplicación se consume vía internet, con administración centralizada, mantenimiento continuo y escalabilidad por demanda (Amazon Web Services, 2026).

Autorespuesta. Mecanismo automatizado que genera respuestas predefinidas o basadas en reglas ante consultas frecuentes, sin intervención humana directa en cada mensaje (IBM, 2026).

API (Application Programming Interface). Conjunto de reglas y contratos que permite la comunicación entre sistemas o componentes de software (WhiskeySockets, 2026).

IA conversacional. Aplicación de técnicas de inteligencia artificial para interpretar lenguaje humano y sostener interacciones más naturales en canales digitales (IBM, 2026).

PLN (Procesamiento del Lenguaje Natural). Área de la IA orientada a analizar, comprender y generar lenguaje humano en texto o voz (Vega Pacheco, 2023).

ORM (Object-Relational Mapping). Enfoque que facilita la interacción con bases de datos relacionales mediante objetos del lenguaje de programación (Drizzle Team, 2026).

Escalabilidad. Capacidad de un sistema para mantener rendimiento adecuado cuando aumenta la carga de usuarios, datos o transacciones (Amazon Web Services, 2026).

Trazabilidad. Capacidad de registrar y seguir el historial de eventos, mensajes y decisiones dentro del sistema (Git Project, 2026).

Kanban. Metodología ágil de gestión visual del trabajo que organiza las actividades en un flujo continuo mediante tableros, columnas y tarjetas. Su aplicación permite visualizar el estado de las tareas, limitar el trabajo en curso e identificar cuellos de botella para mejorar de manera progresiva la entrega de valor. Este enfoque resulta adecuado para proyectos con prioridades cambiantes y equipos pequeños, porque favorece la adaptación continua sin requerir iteraciones rígidas (Anderson, 2010).

Mensajería instantánea. Comunicación digital en tiempo casi real entre usuarios, generalmente por aplicaciones móviles y web (Zarabanda Gelves, 2024).

Tecnologías empleadas

AWS. Servicios de infraestructura y nube para despliegue, escalabilidad y operación del sistema (Amazon Web Services, 2026).

Baileys (API de WhatsApp). Librería para integración y gestión de mensajería sobre WhatsApp Web (WhiskeySockets, 2026).

Drizzle ORM. ORM para TypeScript orientado a tipado fuerte y acceso estructurado a bases de datos SQL (Drizzle Team, 2026).

Git. Sistema de control de versiones distribuido para trazabilidad de cambios y trabajo colaborativo (Git Project, 2026).

GitHub. Plataforma de alojamiento y colaboración de repositorios Git, útil para gestión de código y flujo de trabajo (GitHub, 2026).

MariaDB. Motor de base de datos relacional usado para persistencia de información del sistema (MariaDB Foundation, 2025).

GNU/Linux. Sistema operativo ampliamente utilizado en servidores y entornos de despliegue para aplicaciones web y servicios backend (The Linux Kernel Organization, 2026).

Neovim (nvim). Editor de texto extensible utilizado en flujos de desarrollo de software para edición eficiente de código (Neovim Contributors, 2026).

Node.js. Entorno de ejecución de JavaScript del lado del servidor para construir servicios y APIs de aplicaciones web (Node.js Contributors, 2026).

pnpm. Gestor de paquetes para ecosistema JavaScript/TypeScript optimizado para eficiencia en dependencias (pnpm, 2026).

Penpot. Herramienta *open source* para diseño y prototipado colaborativo de interfaces de usuario (Penpot, 2026).

ReactJS. Biblioteca para construcción de interfaces de usuario reactivas en aplicaciones web (Meta Open Source, 2026).

Taiga. Herramienta *open source* para gestión de proyectos bajo enfoques ágiles (Taiga Agile, 2026).

Tailwind CSS. Framework de utilidades CSS para construir interfaces web de forma rápida y consistente (Tailwind Labs, 2024).

TypeScript. Lenguaje que extiende JavaScript con tipado estático para mejorar mantenibilidad y robustez (TypeScript Team, 2026).

Vite. Herramienta de construcción y desarrollo frontend enfocada en rapidez y experiencia de desarrollo (Vite, 2026).

Visual Studio Code (VS Code). Editor de código multiplataforma con extensiones para desarrollo frontend, backend y depuración de aplicaciones (Microsoft, 2026).

Análisis

El presente capítulo tiene como finalidad identificar, analizar y definir los requerimientos necesarios para el desarrollo del sistema propuesto.

Requerimientos funcionales

- Permitir la conexión e integración con plataformas de mensajería como WhatsApp y Telegram.
- Envío de mensajes (texto, multimedia y mensajes de estado).
- Administración mediante panel web.
- Creación y personalización de cuentas.
- Visualización del historial de conversaciones.
- Actualización dinámica de respuestas sin necesidad de reiniciar el sistema.
- Gestión de múltiples conversaciones simultáneamente.
- Integración con APIs externas.

Requerimientos no funcionales

- **Rendimiento:** procesamiento y respuesta en tiempos reducidos.
- **Disponibilidad:** operación continua minimizando interrupciones.
- **Usabilidad:** interfaz intuitiva para usuarios sin conocimientos técnicos.
- **Escalabilidad:** soporte ante incremento de usuarios y volumen de mensajes.
- **Seguridad:** protección de datos y control de acceso.
- **Confiabilidad:** correcta entrega y almacenamiento de mensajes.
- **Mantenibilidad:** modificaciones sin afectar el funcionamiento.

Reglas de negocio

- El sistema procesa mensajes entrantes y determina respuesta automática solo cuando existe coincidencia con configuraciones previas.
- Las respuestas automáticas son configurables desde el panel web.
- Se registran todos los mensajes recibidos y enviados para trazabilidad.
- Se prioriza una única respuesta por mensaje para evitar duplicidad.
- El envío y recepción respeta las limitaciones de WhatsApp y Telegram.
- Las configuraciones se actualizan sin interrumpir el servicio.

Actores del sistema

- **Usuario:** entidad que interactúa con el producto.
- **Cliente:** usuario que adquiere el producto.
- **Desarrollador:** usuario avanzado que utiliza la API.
- **Administrador:** usuario con permisos avanzados de gestión.

Casos de uso principales

- **Inicio de sesión en WhatsApp:** sincronización mediante código QR o código de vinculación.
- **Integración con Telegram:** configuración mediante API Key de BotFather.
- **Recepción y respuesta:** el sistema recibe mensajes, analiza contenido y genera respuesta automática si hay coincidencia.
- **Envío desde panel web:** comunicación manual a contactos, grupos o listas.

Alcance del MVP

- Conexión a plataformas de mensajería.
- Envío de mensajes.
- Autorespuestas personalizadas.
- Panel de administración web.
- Herramientas para desarrolladores.

Restricciones y supuestos

Restricciones:

- Dependencia del funcionamiento de plataformas externas (WhatsApp y Telegram).
- Limitaciones de tiempo y recursos del proyecto.

Supuestos:

- Usuarios cuentan con acceso a internet.
- Las plataformas externas estarán disponibles.
- El sistema operará inicialmente con volumen moderado de mensajes.

Criterios de aceptación

El sistema deberá permitir conexión exitosa con WhatsApp y Telegram, garantizando envío y recepción de mensajes en tiempo real. También deberá ejecutar respuestas automáticas conforme a las configuraciones establecidas, disponer de un panel web funcional para administración y garantizar el almacenamiento del historial de mensajes.

Riesgos técnicos y operativos

- **Técnicos:** vulnerabilidades y posible pérdida de información.
- **Operativos:** interrupción del servicio y cambios en herramientas externas sin notificación.

Metodología de desarrollo y validación

Enfoque metodológico general

El proyecto adopta Kanban como metodología principal para gestionar el trabajo técnico y Lean Startup como enfoque complementario para validar decisiones de desarrollo mediante evidencia. Bajo esta combinación, cada funcionalidad relevante del MVP se trata como una unidad de trabajo que debe avanzar en el tablero, producir métricas y generar aprendizaje verificable.

En consecuencia, el proyecto no se limita a describir qué se implementó, sino también cómo se gestionó el flujo de trabajo, cómo se definieron hipótesis previas al desarrollo y cómo se obtuvieron aprendizajes útiles a partir de la validación técnica y funcional del sistema.

Kanban como metodología principal

Kanban organiza el desarrollo mediante flujo continuo y visualización del trabajo en curso. Este enfoque resulta adecuado para el proyecto porque permite gestionar tareas técnicas con distintos niveles de

complejidad sin depender de iteraciones rígidas ni de ceremonias fijas, facilitando la priorización continua y la adaptación a incidencias o ajustes durante la construcción del MVP.

Además, Kanban permite observar el estado real de cada actividad, controlar el avance entre etapas y reducir bloqueos mediante seguimiento explícito del trabajo pendiente, en ejecución, en pruebas y finalizado.

Estructura del tablero Kanban en Taiga

El tablero de trabajo se implementa en Taiga con las columnas **New**, **Ready**, **In Progress**, **Ready for Test**, **Done** y **Archived**. Cada tarea debe contar con una finalidad clara, criterio de aceptación y responsable antes de pasar a ejecución efectiva.

El trabajo se distribuye entre los *swimlanes* **Panel Web**, **Motor**, **API**, **Pruebas** y **Crítico**. Esta organización permite seguir las actividades por componente técnico y, al mismo tiempo, distinguir entre desarrollo normal, validación funcional e incidencias que requieren tratamiento prioritario.

Integración de Lean Startup

Lean Startup se incorpora como un marco complementario orientado a validar si las funcionalidades desarrolladas aportan valor al producto. En este proyecto, dicha validación no depende de entrevistas externas, sino de pruebas técnicas, validaciones funcionales y observación del comportamiento del sistema en escenarios controlados.

Su aporte principal consiste en introducir una lógica de aprendizaje dentro del desarrollo: antes de construir una funcionalidad se formula una hipótesis, posteriormente se implementa una versión mínima, luego se mide su comportamiento y finalmente se decide si conviene perseverar o pivotar.

Ciclo Construir-Medir-Aprender

El ciclo *Construir-Medir-Aprender* se aplica sobre funcionalidades concretas del MVP. En la fase de construir se implementa la mínima versión funcional capaz de poner a prueba una hipótesis. En la fase de medir se observan indicadores técnicos, funcionales y de flujo. En la fase de aprender se analizan los resultados para determinar si la solución satisface la necesidad planteada o si requiere ajustes.

Este ciclo se ejecuta dentro del flujo Kanban: la construcción ocurre durante **In Progress**, la observación y verificación técnica durante **Ready for Test** y la decisión de cierre o reajuste antes del paso a **Done** o al carril **Crítico**.

Hipótesis de valor y crecimiento

Antes de desarrollar una funcionalidad, se definen hipótesis de valor y de crecimiento. La hipótesis de valor permite establecer si la funcionalidad resuelve de manera efectiva una necesidad del sistema o mejora una capacidad crítica del producto. La hipótesis de crecimiento, por su parte, determina si la solución

propuesta facilita la expansión del sistema, la reutilización de componentes o la futura incorporación de nuevas capacidades.

Esta formulación previa orienta el desarrollo hacia decisiones justificadas y no únicamente hacia la implementación técnica de requerimientos.

Métricas Kanban-Lean

La evaluación del progreso combina métricas de flujo y métricas de aprendizaje. Entre las primeras se consideran indicadores como *Lead Time*, *Cycle Time*, cantidad de retrabajos y número de incidencias por funcionalidad. Entre las segundas se consideran validación de hipótesis, estabilidad funcional, porcentaje de criterios de aceptación cumplidos y cantidad de decisiones de perseverancia o pivote derivadas de la evidencia.

Estas métricas son accionables porque permiten identificar con mayor precisión dónde se encuentra una dificultad: si en la gestión del flujo, en la implementación técnica o en la definición de la propia funcionalidad.

Registro de experimentos e incidencias críticas

Como parte del enfoque de aprendizaje, el proyecto incorpora un registro de experimentos asociado especialmente al carril **Crítico**. Allí se documentan incidencias relevantes, la hipótesis afectada, la causa probable, el experimento aplicado, el resultado observado y el aprendizaje obtenido.

De esta forma, cada problema importante deja de ser únicamente un error de implementación y se convierte en una oportunidad de mejora del producto y del proceso de desarrollo.

Criterios para pivotar o perseverar

Se persevera cuando una funcionalidad valida su propósito, cumple criterios de aceptación y presenta estabilidad suficiente para mantenerse en el MVP. En cambio, se considera un pivote cuando la funcionalidad conserva valor potencial, pero el enfoque técnico o el alcance definido requieren cambios para responder mejor a los resultados observados.

En este proyecto, pivotar no implica necesariamente descartar una función completa; puede significar simplificar su implementación, redistribuir responsabilidades entre módulos o redefinir su prioridad dentro del producto.

Diseño

Arquitectura del sistema

El sistema utiliza una arquitectura cliente-servidor compuesta por tres componentes principales: frontend, backend y base de datos.

- **Frontend:** aplicación web basada en Preact + Vite para la interfaz de usuario.

- **Backend:** sistema desarrollado en Node.js que procesa solicitudes, aplica lógica de negocio y gestiona comunicación con servicios externos.
- **Base de datos:** almacenamiento de información relacionada con mensajes, conversaciones y configuraciones.

La integración con WhatsApp y Telegram se realiza a través del backend como intermediario. La comunicación frontend-backend usa API REST con JSON sobre HTTP/HTTPS.

Autenticación y seguridad

El sistema utiliza JWT (JSON Web Token) para autenticación. El backend genera un token firmado que el frontend envía en cada petición protegida mediante el encabezado `Authorization: Bearer <token>`. Se valida firma, vigencia y permisos antes de procesar solicitudes.

Principios de diseño

- **Separación de responsabilidades:** componentes independientes (frontend, backend, base de datos).
- **Modularidad:** organización en módulos funcionales.
- **Reutilización:** componentes compartibles para reducir duplicidad.

Convenciones de código

- **Archivos:** snake_case (ej: `get_user.validator.ts`)
- **Funciones/métodos:** camelCase (ej: `getUserById`)
- **Clases/interfaces:** PascalCase (ej: `UserValidator`)

Configuración

Variables de entorno en archivo `.env` para separar parámetros sensibles (credenciales, tokens, claves). Se excluyen del versionado con `.gitignore`. Gestión mediante `node-env` por entorno (desarrollo, pruebas, producción).

Control de versiones

Git + GitHub con flujo GitFlow: `main` (versiones estables), `develop` (integración continua), `feature/*`, `release/*`, `hotfix/*`.

Arquitectura backend

El backend utiliza la arquitectura hexagonal (puertos y adaptadores) ampliada para mayor modularidad y desacoplamiento.

Arquitectura hexagonal. Separa la aplicación en capas con responsabilidad independiente, logrando desacoplamiento y reutilización:

- **Puerto:** interfaz/contrato que define QUÉ operaciones están disponibles.
- **Adaptador:** implementación concreta que conecta con algo externo.

Modificaciones implementadas.

1. **Organización módulo-primero:** se agrupa primero por módulo y dentro las capas.
2. **API como módulo adaptador de alto nivel:** `src/api/` como directorio de primer nivel con estructura MVC.
3. **Infraestructura dual:** compartida transversal y específica por módulo.
4. **Simplificación:** sin casos de uso formales ni puertos/interfaces formales.
5. **Punto de entrada dual:** servidores WebSocket (WhatsApp) y HTTP REST en paralelo.

Estructura de directorios.

```
src
|-- api/                # Módulo adaptador HTTP
|-- config/
|-- infrastructure/     # Base de datos, logging, etc.
|-- modules/           # WhatsApp, Telegram. etc.
|-- types/              # Tipos de datos personalizados.
```

Tipos de componentes.

- **Adaptador primario:** `src/api/` - recibe peticiones HTTP.
- **Adaptador secundario:** repositorios - conecta con MySQL.
- **Módulo de dominio:** `src/modules/whatsapp/` - lógica de negocio.
- **Servicio de aplicación:** punto de entrada a lógica de negocio.

Reglas de dependencia.

- Adaptadores **dependen** de servicios y repositorios.
- Servicios **dependen** de repositorios, nunca de adaptadores.
- Repositorios **dependen** del ORM y schemas, nunca de adaptadores.
- Dominio no tiene dependencias externas.

Topología de despliegue

El entorno `develop` usa AWS EC2 `t4g.small` (ARM64, 2 vCPU, 2 GiB, RHEL) como nodo único para el MVP, Mientras que el entorno de Producción usará AWS Elastic Beanstalk.

Entornos de ejecución

- **Develop:** EC2 + RHEL + PM2 para operación continua y reinicio automático.
- **Producción:** Elastic Beanstalk con mismo artefacto.

El software se construye (transpilacion optimizada) con `esbuild` para optimizar tiempos de arranque.

Escalabilidad

- **Develop:** escalado vertical sobre EC2.
- **Producción:** escalado automático horizontal con balanceador de carga.

Seguridad

Conectividad pública mediante IPv6, complementada con reglas del grupo de seguridad en AWS.

Monitoreo y recuperación

- **Monitoreo:** PM2 en develop, Elastic Beanstalk en producción.
- **Respaldo:** recuperación vía reprovisionamiento de instancia y restauración de datos.

Diseño de base de datos

La base de datos está diseñada para almacenar mensajes, usuarios, conversaciones y configuraciones.

Utiliza Drizzle ORM con enfoque relacional y migraciones versionadas.

Modelo de datos

- **Conceptual:** gestión con Drizzle ORM de manera evolutiva.
- **Lógico:** schemas con tablas, relaciones y reglas de integridad.
- **Físico:** migraciones con identificadores UUID para unicidad global.

Convenciones de datos

- Claves primarias en `uuid`.
- Claves foráneas con prefijo `id_`.
- Tablas y columnas en `snake_case`.
- Índices en campos de búsqueda frecuente.

Las migraciones se aplican de forma incremental durante el desarrollo, con respaldo previo en entornos productivos.

Diseño de API e integraciones

El sistema integra canales externos mediante capa API que desacopla la interfaz de usuario de los servicios de mensajería. Los contratos de comunicación usan JSON con operaciones delimitadas para CRUD.

Endpoint	Método	Descripción
/api/enviar	POST	Envío de mensajes
/api/recibir	POST	Recepción de mensajes
/api/historial	GET	Consulta de conversaciones

Cuadro 1

Endpoints principales de la API

Diseño de experiencia de usuario

Las pantallas priorizan claridad en estados, retroalimentación inmediata, mensajes de error interpretables y continuidad entre acciones. Procesos complejos como vinculación de canales, administración de respuestas o seguimiento de conversaciones se presentan mediante interacciones simples y previsibles.

Desarrollo

Estrategia de desarrollo incremental

El desarrollo del MVP de manera incremental, prioriza funcionalidades que habilitan capacidades centrales del sistema. Bajo este enfoque, cada funcionalidad se implementa como una unidad de valor que puede involucrar a varios módulos del sistema, pero que se documenta como una sola capacidad operativa dentro del producto.

Funcionalidades del MVP

En este capítulo, el desarrollo se organiza por funcionalidades del MVP. Cada una se documenta como una capacidad operativa del producto, aunque su implementación técnica involucre a varios módulos del sistema.

Validación, métricas y aprendizaje

Estrategia de validación técnica y funcional

La validación del proyecto se realiza en un entorno controlado por el equipo de desarrollo, verificando tanto el comportamiento técnico de los módulos como la coherencia funcional de las capacidades implementadas en el MVP. Bajo este enfoque, una funcionalidad se considera validada cuando puede ejecutarse de manera

estable, cumple sus criterios de aceptación y genera evidencia suficiente para decidir si conviene perseverar o realizar ajustes.

Registro de experimentos y aprendizajes

Discusión de resultados

Conclusiones y recomendaciones

Conclusiones generales

Aportes metodológicos del proyecto

Limitaciones del trabajo

Recomendaciones

Líneas de trabajo futuro

Referencias

- Alia, P. A., Febriana, R. W., Prayogo, J. S., & Kriswibowo, R. (2024). Implementation Chatbot on Whatsapp Using Artificial Intelligence With Natural Language Processing Method. *ELECTRON Jurnal Ilmiah Teknik Elektro*, 5(1). <https://doi.org/10.33019/electron.v5i1.134>
- Amazon Web Services. (2026). *AWS Documentation* [Documentación oficial consultada en línea]. <https://docs.aws.amazon.com/>
- Anderson, D. J. (2010). *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press.
- Cockburn, A. (2005). *Hexagonal architecture* [Artículo original sobre arquitectura hexagonal]. Consultado el 31 de marzo de 2026, desde <https://alistair.cockburn.us/hexagonal-architecture/>
- Drizzle Team. (2026). *Drizzle ORM Documentation* [Documentación oficial consultada en línea]. <https://orm.drizzle.team/>
- Elsen, R., & Muhamad, Y. (2025). Development of Web-based Hijab Product E-Catalog Application Integrated with WhatsApp Chatbot [Submitted Jul 23, 2025; Accepted Sep 12, 2025; Published Nov 30, 2025]. *Jurnal Sistemantik*, 1(2). <https://doi.org/10.33364/sistemantik/v.1-2.2536>
- Git Project. (2026). *Git Documentation* [Documentación oficial consultada en línea]. <https://git-scm.com/doc>
- GitHub. (2026). *GitHub Docs* [Documentación oficial consultada en línea]. <https://docs.github.com/>
- IBM. (2026). *What is a chatbot?* [Consultado en línea]. <https://www.ibm.com/think/topics/chatbots>
- Lino Cruz, J. R., & Paladines Morán, J. N. (2023). *Implementación de una plataforma de chatbot gestionado por WhatsApp y Telegram para su comercialización en la empresa Innovus Software* [Tesis de Ingeniería en Tecnología de la Información]. Universidad Estatal del Sur de Manabí. <http://repositorio.unesum.edu.ec/handle/53000/4767>
- MariaDB Foundation. (2025). *MariaDB Server — Open Source Database*. <https://mariadb.org/>
- Meta Open Source. (2026). *React Documentation* [Documentación oficial consultada en línea]. <https://react.dev/>
- Microsoft. (2026). *Visual Studio Code Documentation* [Documentación oficial consultada en línea]. <https://code.visualstudio.com/docs>
- Neovim Contributors. (2026). *Neovim Documentation* [Documentación oficial consultada en línea]. <https://neovim.io/doc/>
- Node.js Contributors. (2026). *Node.js Documentation* [Documentación oficial consultada en línea]. <https://nodejs.org/docs/latest/api/>

- Penpot. (2026). *Penpot Documentation* [Documentación oficial consultada en línea].
<https://help.penpot.app/>
- Pest PHP Team. (2024). *Pest PHP Testing Framework*. <https://pestphp.com/>
- pnpm. (2026). *pnpm Documentation* [Documentación oficial consultada en línea]. <https://pnpm.io/>
- Salguero, E. (s.f.). *Arquitectura Hexagonal* [Publicación en Medium, consultada en línea]. Consultado el 31 de marzo de 2026, desde <https://medium.com/@edusalguero/arquitectura-hexagonal-59834bb44b7f>
- Singh, A. (2021). Baileys: Multi-device WhatsApp Web API [Originally developed for CS-2362 at Ashoka University]. <https://github.com/WhiskeySockets/Baileys>
- Suárez Rosas, C. A. (2023). *Prototipo de chatbot con inteligencia artificial* [Tesis de Maestría en Tecnologías de la Información]. Tecnológico de Estudios Superiores de Cuautitlán Izcalli, División Posgrado [Director: Fernando René Barbosa Morales; fecha: 25 de octubre de 2023].
<https://rinacional.tecnm.mx/bitstream/TecNM/6799/2/TESIS%20alberto%20Su%C3%A1rez.pdf>
- Taiga Agile. (2026). *Taiga Documentation* [Documentación oficial consultada en línea].
<https://docs.taiga.io/>
- Tailwind Labs. (2024). *Tailwind CSS Documentation*. <https://tailwindcss.com/>
- The Linux Kernel Organization. (2026). *Linux Kernel Documentation* [Documentación oficial consultada en línea]. <https://docs.kernel.org/>
- The PHP Group. (2024). *PHP: Hypertext Preprocessor* [Versión 8.4.14]. <https://www.php.net/>
- TypeScript Team. (2026). *TypeScript Documentation* [Documentación oficial consultada en línea].
<https://www.typescriptlang.org/docs/>
- Vargas, G. d. S. d., & Braga, M. L. (s.f.). Implementação de assistente virtual via WhatsApp para aprimorar a comunicação no IFC – Campus Sombrio [Trabalho técnico em fase de implementação e testes].
https://www.researchgate.net/profile/Matheus-Braga-2/publication/368385709_Implementacao_de_Assistente_Virtual_via_WhatsApp_para_aprimorar_a_comunicacao_no_IFC_-_Campus_Sombrio/links/63e4f50dc002331f7266bd63/Implementacao-de-Assistente-Virtual-via-WhatsApp-para-aprimorar-a-comunicacao-no-IFC-Campus-Sombrio.pdf
- Vega Pacheco, I. A. (2023). *Procesamiento de lenguaje natural con modelos inteligentes para la atención de consultas mediante redes sociales* [Tesis de grado]. Universidad Mayor de San Andrés, Facultad de Ciencias Puras y Naturales, Carrera de Informática [Repositorio institucional UMSA].
<https://repositorio.umsa.bo/xmlui/handle/123456789/42678>
- Vite. (2026). *Vite Documentation* [Documentación oficial consultada en línea]. <https://vite.dev/guide/>

WASP Foundation. (2025). *OWASP — Open Web Application Security Project*.

<https://owasp.org/www-project-top-ten/>

WhiskeySockets. (2026). *Baileys Documentation* [Documentación oficial consultada en línea].

<https://baileys.wiki/>

Zarabanda Gelves, E. H. (2024). Desarrollo de un chatbot, con la API de WhatsApp, que permita la gestión de propuestas de trabajo de grado (anteproyectos), en la facultad tecnológica de la Universidad Distrital Francisco José de Caldas.

<https://repository.udistrital.edu.co/server/api/core/bitstreams/785f0f2c-8dd5-4326-bd7a-ffa154a47228/content>

Apéndice

Anexos

Diagrama de Gantt en PlantUML

```

@startgantt
language es
Project starts 2026-02-23
printscale weekly

-- ASPECTOS INICIALES --
[Desarrollo del perfil] as [AJU] lasts 1 week
[Revisión bibliográfica] as [REV] lasts 2 weeks

-- ANÁLISIS --
[Análisis de requerimientos] as [REQ] lasts 2 weeks
[REQ] starts at [REV]'s end

-- DISEÑO --
[Diseño de arquitectura] as [ARQ] lasts 2 weeks
[Diseño de estructuras de datos] as [DB] lasts 2 weeks
[ARQ] starts at [REQ]'s end
[DB] starts at [REQ]'s end

-- DESARROLLO E INTEGRACIÓN --
[Configuración del entorno] as [ENT] lasts 1 week
[ENT] starts at [ARQ]'s end
[Desarrollo backend] as [BACK] lasts 4 weeks
[Desarrollo frontend] as [FRONT] lasts 4 weeks
[Desarrollo agentes autorespuesta] as [AGENT] lasts 4 weeks
[BACK] starts at [ENT]'s end
[FRONT] starts at [ENT]'s end
[AGENT] starts at [ENT]'s end

[Integración con WhatsApp] as [WA] lasts 1 week
[Integración con Telegram] as [TG] lasts 1 week
[WA] starts at [BACK]'s end
[TG] starts at [BACK]'s end

-- PRUEBAS --
[Pruebas unitarias e integración] as [PUNIT] lasts 1 week
[Pruebas de validación] as [PVAL] lasts 1 week
[PUNIT] starts at [TG]'s end
[PVAL] starts at [PUNIT]'s end

-- DOCUMENTACIÓN Y CIERRE --
[Elaboración de documentación] as [DOC] lasts 1 week
[Ajustes finales] as [FIX] lasts 1 week
[DOC] starts at [PVAL]'s end
[FIX] starts at [DOC]'s end

-- PREPARACIÓN DEFENSA --
[Preparación de presentación] as [PRES] lasts 1 week
[Pruebas y ajustes finales] as [ENSAYO] lasts 1 week
[PRES] starts at [FIX]'s end
[ENSAYO] starts at [PRES]'s end

-- HITOS --
[Hito: Inicio del proyecto] as [H1] happens at 2026-02-23
[Hito: Entrega final] as [H2] happens at 2026-07-11

@endgantt

```

Diagrama de secuencia en PlantUML para el inicio de sesión de WhatsApp mediante

QR

```
@startuml
actor Usuario
participant "Panel Web" as Panel
participant "API" as API
participant "Motor" as Motor
participant "WhatsApp" as WA

Usuario -> Panel: Solicita vinculación de cuenta
Panel -> API: Iniciar proceso de autenticación
API -> Motor: Crear sesión temporal
Motor -> WA: Solicitar datos de autenticación
WA --> Motor: Datos para QR
Motor --> API: QR + estado inicial
API --> Panel: Mostrar QR
Usuario -> WA: Escanea QR
WA --> Motor: Autenticación exitosa
Motor --> API: Estado = conectado
API --> Panel: Actualizar estado de sesión
Panel --> Usuario: Cuenta vinculada
@enduml
```